

情報ネットワークII

プログラミング課題

2026年度 国際創造工学科 情報系
4年前期

周 而晶
しゅう じしょう

2026年度プログラミング課題

- 概要説明
- サンプルコードについて
- 動作確認用プログラムについて
- 開発環境の準備

この資料は、茨城大学工学部情報工学科米倉達広教授が作成した学部2年生向け「プログラミング演習Ⅲ」の講義資料をもとにして、昨今のBYOD環境に合わせて制作したものです。

概要説明

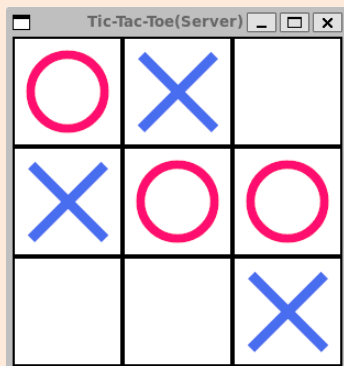


課題内容

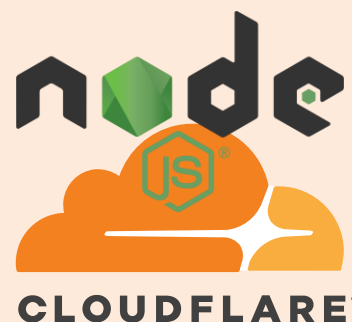
クライアント・サーバ型X11 GUIプログラミング

「Web API版対戦型〇×ゲーム」

Client



Server



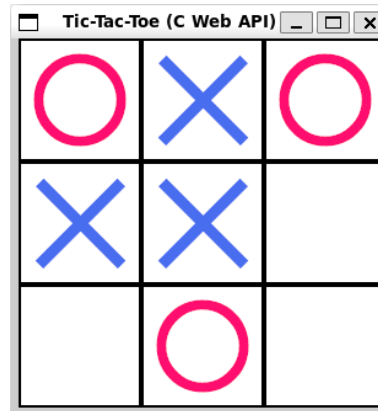
課題内容

Web API版対戦型○×ゲーム (Tic-Tac-Toe)

機能：

- HTTP GET Request(Web API)経由でサーバと対戦できる
- プレイヤはGUIで盤面をクリックして操作する

完成イメージ：



課題内容

プログラム仕様

- 動作環境：X Windowの動作するUNIX系OS
- 開発環境：TCP/IPソケットライブラリ + X11ライブラリ、GCC(Linux/WSL)/Clang(Mac)
- 盤面サイズ： 3×3
- 通信方式：TCP/IPのソケット(コネクション型)を用いる

課題内容

ルール：

プレイヤーは先手(○)、サーバは後手(×)とする。

通信仕様：

盤面(9文字)と自分の学籍番号(7文字)をそれぞれURLパラメータ "board" と "key" に設定し <http://api.islaytouch.com/ttt.php> へGET送信する。

データ形式：

サーバからの応答はJSON形式で、C言語の2次元配列と同じ構造である。

勝敗判定：

サーバではなく、**クライアント側**のプログラムで行う。

課題内容 – データ構造と送受信の手順

プログラムは以下の手順を繰り返してゲームを進行させる。
各段階でデータ形式(型)が変化することに注意。

【変換】2次元配列⇒文字列

現在の盤面データ(2次元配列)を、1行の文字列(9文字)に変換する。

例： $\{\{1, 0, 0\}, \{0, 0, 2\}, \{0, 0, 0\}\} \Rightarrow "100002000"$

【送信】HTTP GETリクエスト

変換した文字列と自分の学籍番号をそれぞれURLパラメータ"board"と"key"に設定して送信する。URL例：

`http://api.islaytouch.com/ttt.php?board=100002000&key=2023000`

【受信】JSONレスポンス

サーバは思考結果を反映した盤面を**JSON形式**で返す。

データ構造はC言語の配列と同じ形式の2次元配列となっている。

【反映】盤面の更新

受け取ったデータを、自分の盤面データに上書きして盤面を更新する。

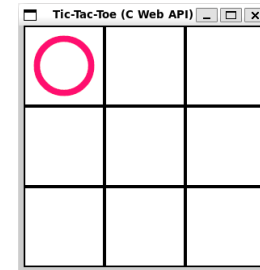
課題内容 – データの具体例 (C(X11 GUI) ⇔ Web API)

例：プレイヤー(先手：○=1)が左上に置いた直後の通信

①送信時(C⇒API)

- C(変換前)：2次元配列(int型)

```
board = {  
    {1, 0, 0},  
    {0, 0, 0},  
    {0, 0, 0}  
}
```



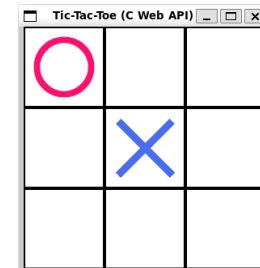
- URLパラメーター(通信時)：

<http://api.islaytouch.com/ttt.php?board=100000000&key=2023000>

②受信時(API⇒C)

- JSON(通信時)：例えば、サーバ(後手：×=2)が中央に置いた場合

```
{  
    "board": [  
        [1, 0, 0],  
        [0, 2, 0],  
        [0, 0, 0]  
    ]  
}
```



- C(変換後)：JSONの中身を解析し、盤面データ配列に上書きする。

評価基準

- ① サーバへのデータ送信
- ② サーバからのデータ受信
- ③ 盤面の**再描画**
 - GUIウィンドウを最小化したり、リサイズしたりした後も盤面の表示が消えない(再び描画される)こと。
- ④ 勝敗判定の実現

評価基準

加対象

- ・ 提出物の①～②がそろっていること +2(各1点×2)
- ・ ソースコードがコンパイルできること +1
- ・ 課題 ①：+10； ②：+10； ③：+5； ④：+5

※要求仕様を完全に満たしている場合の点数（部分点あり）。

減対象

- ・ 他人の成果物・生成AIの書き写し 最終得点より1～24点減点。
※「参考」や「借用」を超えた範疇で、明らかにオリジナリティに欠けている場合のみが対象。

採点基準の詳細

- ・ ソースコードがコンパイルできない場合は、評価の対象外とする。
- ・ ソースコードの提出が確認できなかった場合：「提出物」の件数に応じた評価点のみが加算される。
- ・ 「減対象」に該当し、類似度が90%未満の場合：最終得点より最大50%減点（端数切捨て）。
- ・ 「減対象」に該当し、類似度が90%以上の場合：最終得点より最大80%減点（端数切捨て）。

なお、合計点数が30点以上の場合は課題評価点を満点とする。

課題の提出

提出期限：

8月7日（金） 17:00

Google Classroom経由で提出

提出物：

① ソースコード

- コンパイルできるソースファイル一式。
- PDFやdocxファイルへの貼り付けは不要。

② 実行ファイル

③ 添付資料（対象者のみ）

- 生成AIを使用した場合、チャット履歴を[AI Exporter](#)でPDFに出力して提出する。

注意事項：

- C言語のみを使用すること（C++を含めて他の言語は不可）。
- ビルドツールなどを使用せずに直接gccコマンドでコンパイルすること。
- コンパイル環境にMacOSを使った場合、指定された環境構築方法（XQuartz）以外は認められない（一部採点不能のため）。

サンプルコードについて



サンプルコード

❑ fakegpt.c

チャットAPIクライアントプログラム。

リクエストURL：

`http://api.islaytouch.com/chat.php?msg=[送信メッセージ]`

レスポンス(JSON)：

```
{"msg":"[受信メッセージ]"}
```

❑ dotter.c

ビットマップ画像(XPM)描画処理及びイベントハンドリングなどのロジックが実装されたサンプルプログラム。

➤ none.xpm, circle.xpm, cross.xpm

盤面描画用ビットマップ画像。



🔗 **git**リポジトリ：

<https://github.com/zj13924/Net2>

コンパイル・実行方法

□ fakegpt.c :

コンパイル

> gcc -o fakegpt fakegpt.c *[-Wall]*

実行

直接接続(プロキシなし)の場合

> ./fakegpt

高専プロキシサーバ経由の場合

> ./fakegpt -p

□ dotter.c (+none.xpm, circle.xpm, cross.xpm) :

コンパイル

> gcc -o dotter dotter.c -lX11 -lXpm *[-Wall]*

実行

> ./dotter

※: *[青文字]*は、オプションを意味する。

動作確認用プログラムについて



動作確認用プログラム(実行ファイル)

■ eval

Windows x64 WSL2環境用実行ファイル

■ eval_mac

MacOS Arm64 XQuartz環境用実行ファイル

実行方法

実行権限の付与

> chmod +x eval

実行

直接接続(プロキシなし)の場合

> ./eval

高専プロキシサーバ経由の場合

> ./eval -p

※：[青文字]は、オプションを意味する。

開発環境の準備



開発環境の準備

- Option 1: Windows + WSL2
- Option 2: MacOS + XQuartz
- Option 3: Pure Linux

Option 1: Windows + WSL2

- 管理者権限でWindows PowerShellを開き、下記のコマンドを実行する
 - > `wsl --install`
- インストールが終わったら、PCを再起動する
- スタートメニューから“wsl”を検索し、WSLのコマンドプロンプトを開く
- Linuxのアカウント設定が求められたら、今後WSLの中で使うユーザ名とパスワードを入力する（Windowsのアカウント情報と一致させる必要はない）
- 下記のコマンドを実行し、必要なパッケージをインストールする



```
> sudo echo 'Acquire::http::proxy "http://po.cc.ibaraki-  
ct.ac.jp:3128/";' >> /etc/apt/apt.conf  
> sudo echo 'Acquire::https::proxy "http://po.cc.ibaraki-  
ct.ac.jp:3128/";' >> /etc/apt/apt.conf
```

```
> sudo apt update
```

```
> sudo apt --fix-broken install
```

```
> sudo apt install build-essential -y --fix-missing
```

```
> sudo apt install libx11-dev -y --fix-missing
```

```
> sudo apt install libxpm-dev -y --fix-missing
```

```
> sudo apt install net-tools -y --fix-missing
```

学内ネットワーク環境向けのプロキシ設定

Option 1: Windows + WSL2

- WSLの内部ファイルシステムへのアクセス方法

WSLのコマンドプロンプトで下記のコマンドを実行する

> explorer.exe [*Linuxファイルシステムのパス*]

〔例〕現在のディレクトリを表示

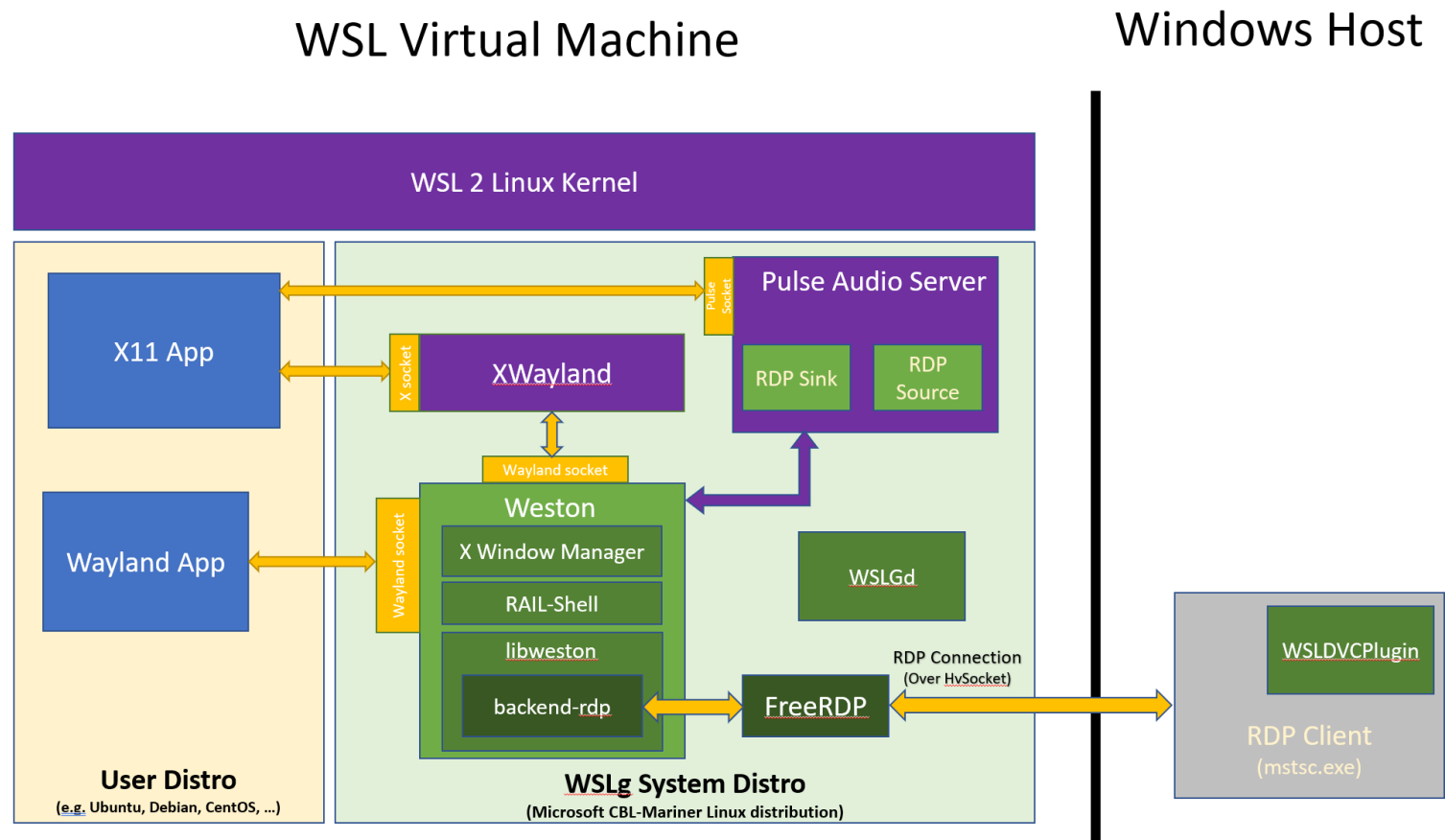
ホームディレクトリへ移動

> cd ~

> explorer.exe .

Option 1: Windows + WSL2

〔参考〕 WSL2の仕組み^[1]



[1] <https://github.com/microsoft/wslg>

Option 2: MacOS + XQuartz

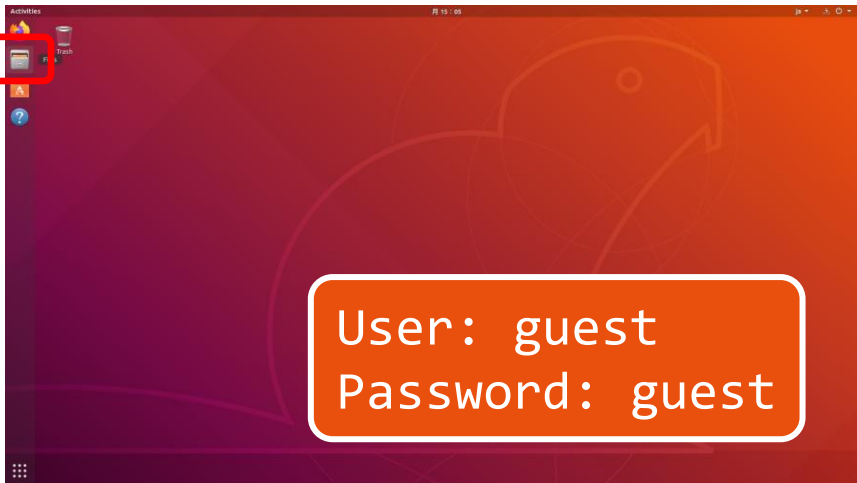


- XQuartzをインストールする
 - <https://www.xquartz.org>よりインストーラ(pkg)をダウンロードしてインストールする
- ターミナルを開き、下記のコマンドを実行する
 - # パッケージマネージャHomebrewのインストール
 - > /bin/bash -c "\$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
 - # 必要なライブラリのインストール
 - > brew install libx11
 - > brew install libxpm
 - # コンパイラ用環境変数の追加 (zshを使う場合)
 - > echo 'export C_INCLUDE_PATH="/opt/homebrew/include"' >> ~/.zshrc
 - > echo 'export LIBRARY_PATH="/opt/homebrew/lib"' >> ~/.zshrc
 - > source ~/.zshrc

Option 3: Pure Linux

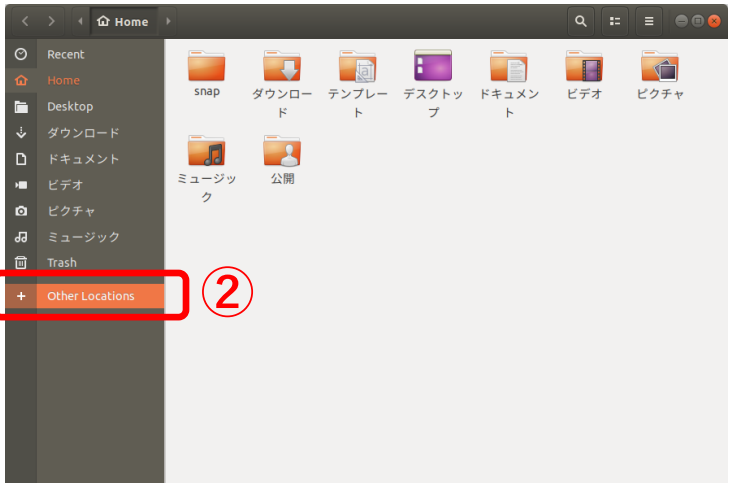
・情報工学演習室の端末でのコンパイル・実行方法

①

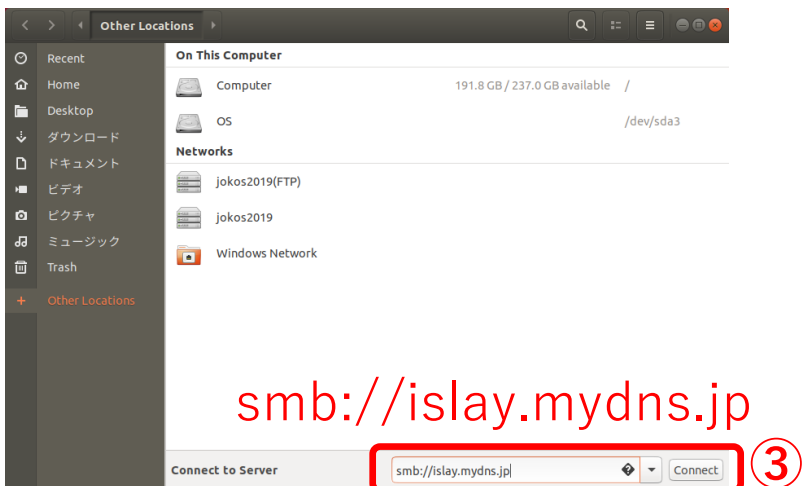


User: guest
Password: guest

②

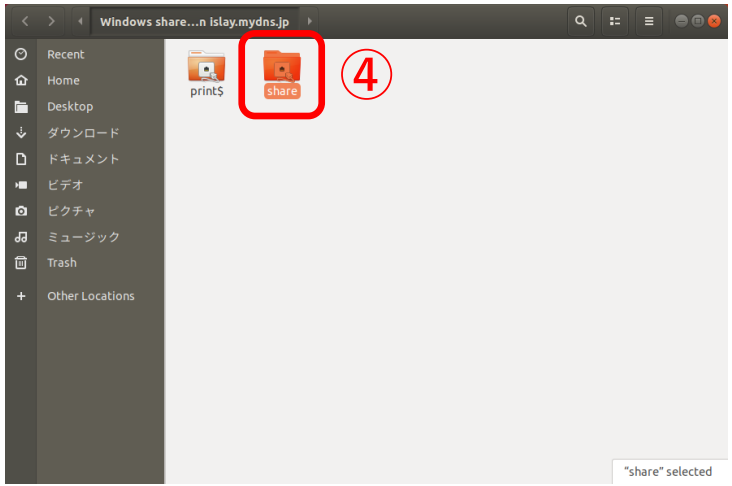


③



smb://islay.mydns.jp

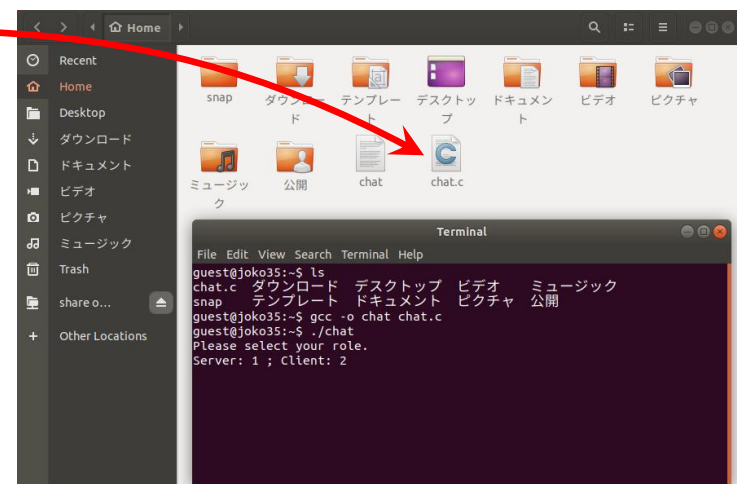
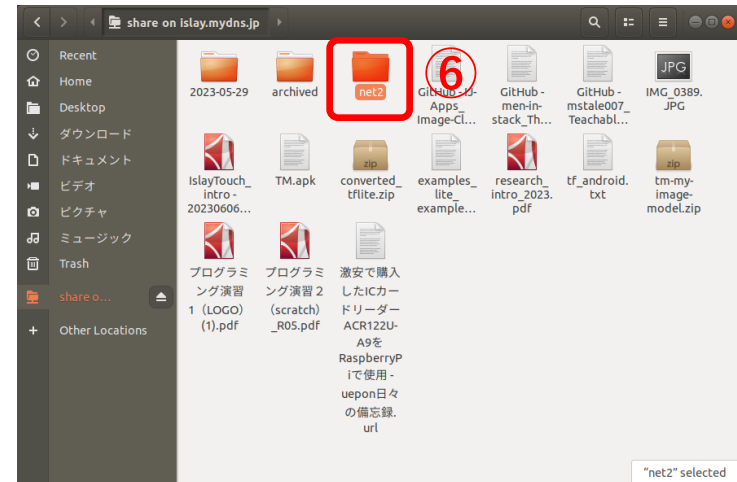
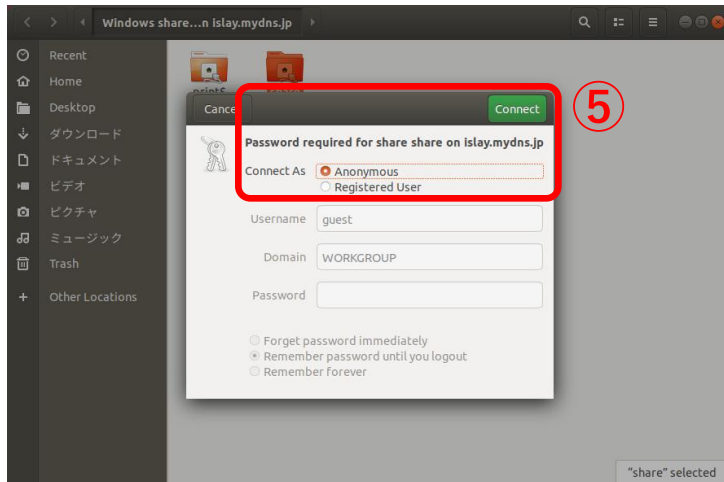
④



print\$ share

Option 3: Pure Linux

・情報工学演習室の端末でのコンパイル・実行方法



Option 3: Pure Linux

• 情報工学演習室の端末でのコンパイル・実行方法

コンパイル済み実行ファイルへの実行権限付与：

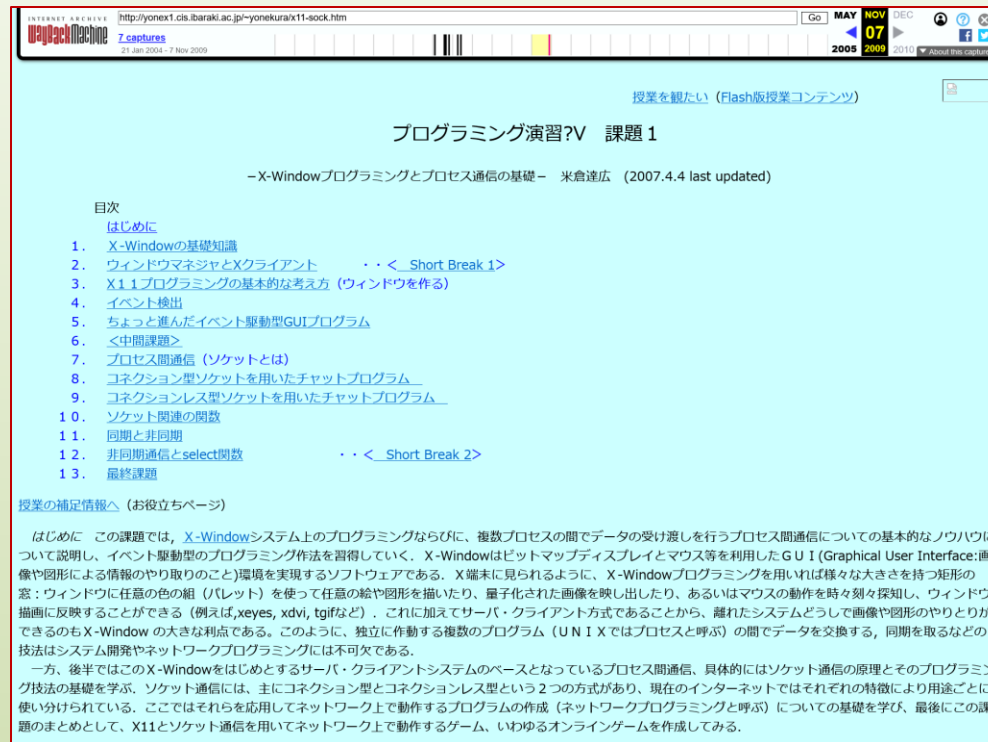
〔例〕

```
> chmod +x eval
```

サンプルコード、動作確認用プログラムの格納先：

- <https://github.com/zj13924/Net2>

参考文献(1)



茨城大学工学部 情報工学科 米倉 達広 教授「プログラミング演習Ⅲ」

情報工学科 学部2年生向け(開講当時)の授業資料

<https://web.archive.org/web/20091107144112/http://yonex1.cis.ibaraki.ac.jp/~yonekura/x11-sock.htm>

参考文献(2)



茨城大学工学部 情報工学科 米倉 達広 教授、鎌田 賢 教授「情報処理概論」
情報工学科 学部2年生向け(開講当時)の授業資料

<https://web.archive.org/web/20150310012736/http://yonex1.cis.ibaraki.ac.jp/jsindex.html>

<https://web.archive.org/web/20150425211627/http://yonex1.cis.ibaraki.ac.jp/Win32/index.html>